

Customer Feedback System – Architectural Design Document

1. Overview

A microservices-based system to collect, process, analyze, and report on customer feedback. The solution uses ASP.NET Core (.NET 6+), Entity Framework Core, React/Angular for the SPA, and PostgreSQL or MSSQL for persistent storage. Async messaging (RabbitMQ/Azure Service Bus/Kafka) decouples analytics and reporting.

2. Functional Requirements Mapping

• Feedback submission form • Categorization and sentiment • Manager landing-page summary • Self-registration & login

3. Architecture Principles

Microservices per bounded context • Synchronous REST for CRUD • Asynchronous events for analytics/reporting • Database-per-service • JWT-based security • Observability via structured logs, metrics, and tracing.

4. Logical Components

Auth Service • Feedback Service • Catalog Service • Analytics Worker • Reporting Service • API Gateway/Ingress • Message Broker • Cache • Databases • Secrets Vault • Observability stack.

5. Data Models (Representative)

Entity	Key Fields
Feedback	id, store_id, customer_id, rating (1-5), comment, category_id, created_at
Sentiment	feedback_id, sentiment_score, sentiment_label, processed_at
Category	id, name, description, is_active
User	id, email, password_hash, roles, store_id

6. APIs (Representative)

Auth: /register, /login, /refresh • Feedback: POST/GET/PUT/DELETE • Reporting: /summary, /distribution • Catalog: /categories • Analytics: /reprocess/{id}

7. Security

JWT auth • Role-based policies • HTTPS-only • Security headers • Secrets in Vault • Minimal PII • Encryption at rest

8. Validation & Exception Handling

FluentValidation • server-side + client-side checks • global exception handlers • RFC7807 ProblemDetails responses

9. Logging & Observability

Serilog structured logs • OpenTelemetry traces • Prometheus metrics • Health checks /live /ready • Correlation IDs

10. Performance & Scalability

Indexes • pagination • EF optimization • Redis caching • GZIP • HTTP/2 • Polly retries • HPA autoscaling • Materialized views for reporting

11. Testing & Code Quality

xUnit unit tests • Moq/NSubstitute • Integration tests • UI tests • k6 performance tests • SonarQube static analysis

12. Configuration

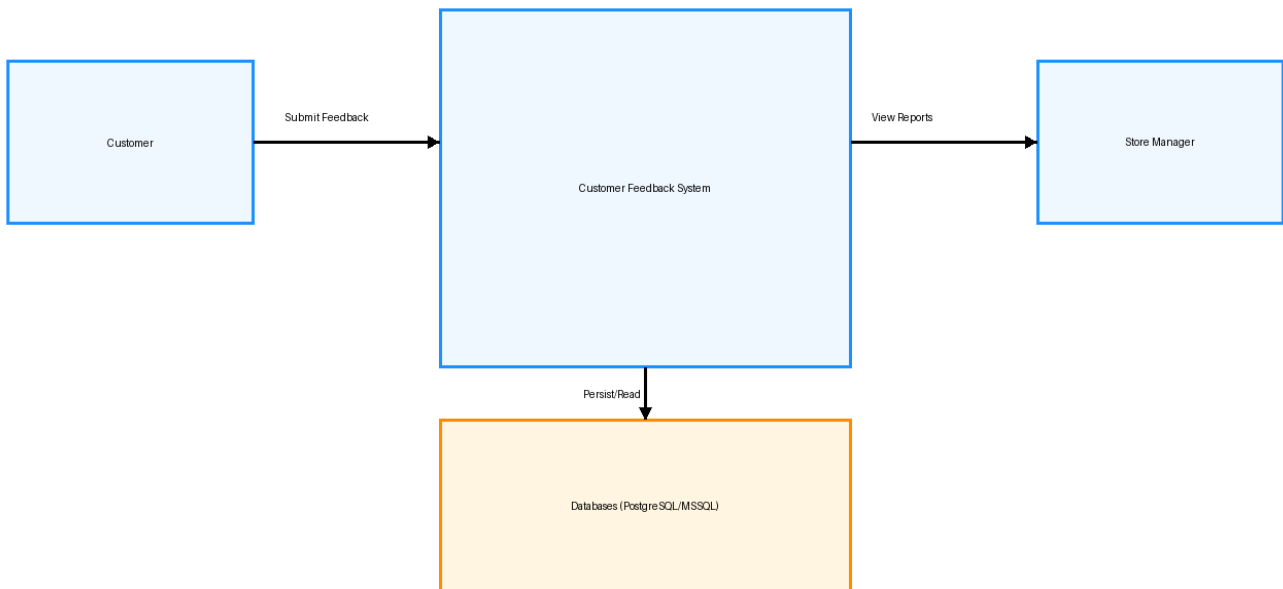
appsettings.json per env • environment variables • feature flags • encrypted sensitive data • Key Vault integration

13. Deployment & DevOps

Docker containers • Kubernetes (AKS/EKS) • Ingress • TLS termination • Helm charts • GitHub Actions/Azure DevOps CI/CD • Blue/green or canary releases

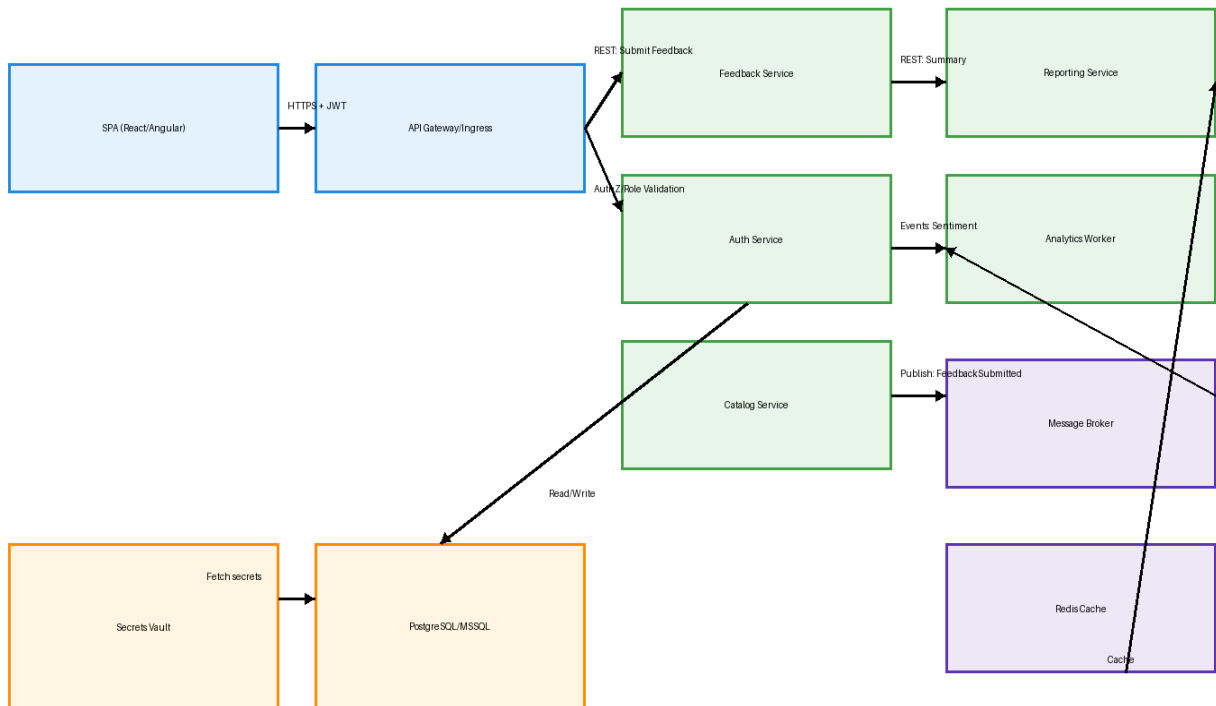
14. Data Flow Diagram – Level 0 (Context)

Customer Feedback System Data Flow (Level 0)



Legend: External entities (Customer, Store Manager), central system boundary, and primary data stores with main flows.

15. Data Flow Diagram – Level 1 (Internal Flows)



Legend: SPA, API Gateway/Ingress, microservices, message broker, cache, databases, and secrets vault with synchronous and asynchronous flows.